Side Activity 4: Advanced Data Integrity & Constraints

CSCI3170 Introduction to Database Systems

Learning Objectives

In Side Activity 3, you learned how to create basic tables. In this Side Activity, we focus on **Data Quality** and **Relationships**. You will learn:

- How to enforce complex logic using **CHECK** constraints (e.g., "Salary must be positive").
- How to manage relationships using **FOREIGN KEY** constraints.
- The critical difference between **ON DELETE CASCADE** and **ON DELETE SET NULL**.
- How to **modify existing tables** (BASIC operations) using **ALTER TABLE**.

Part 0: Getting started

Please refer to the instructions in **Side Activity 3** to access your Oracle environment.

Part 1: Logic Constraints (CHECK)

Constraints like `NOT NULL` are simple. But what if you need to enforce business logic? The `CHECK` constraint allows you to write a mini-expression that every row must pass.

Step 1: Single Condition Check

Let's create a table for a company. We want to ensure that the salary is strictly greater than 0.

```
CREATE TABLE company_v3 (
    id INTEGER PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    salary REAL CHECK(salary > 0)
);
```

Test it: Try to insert a negative salary.

```
INSERT INTO company_v3 VALUES (1, 'Evil␣Corp', -500);
```

Result: You should see an error: `CHECK constraint failed: salary > 0`.

Step 2: Multiple Logic Checks

You can combine conditions using `AND` / `OR`. Let's create a product table where: 1. The list price must be greater than or equal to the discount. 2. Both values must be positive.

```
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name VARCHAR(50) NOT NULL,
    list_price DECIMAL(10, 2) NOT NULL,
    discount DECIMAL(10, 2) DEFAULT 0,
    CHECK (list_price >= 0 AND discount >= 0 AND list_price >= discount)
);
```

Part 2: Foreign Keys & Actions

The Scenario

We have **Companies** (Parent) and **Items** (Child).

- **Parent Table:** `company_v4`
- **Child Table:** `item`

Step 1: Create the Parent

First, create the company table and add some data.

```
CREATE TABLE company_v4 (
    id INTEGER PRIMARY KEY,
    name VARCHAR(50) NOT NULL
);

INSERT INTO company_v4 VALUES (1, 'Apple');
INSERT INTO company_v4 VALUES (2, 'Samsung');
```

Step 2: The Default Behavior (No Cascade)

Create the child table with a **standard** Foreign Key (without cascade). This is the default safety setting in databases.

```
CREATE TABLE item (
    item_id INTEGER PRIMARY KEY,
    item_desc VARCHAR(50) NOT NULL,
    com_id INTEGER,
    -- Standard Foreign Key
    FOREIGN KEY (com_id) REFERENCES company_v4(id)
);

-- Insert an iPhone linked to Apple (ID 1)
INSERT INTO item VALUES (101, 'iPhone', 1);
```

Step 3: Attempt to Delete (The Error)

Try to delete "Apple" from the company table while it still has an item linked to it.

```
DELETE FROM company_v4 WHERE id = 1;
```

Observation: You should see an error:
`ORA-02292: integrity constraint ... violated - child record found.`
Why? The database is protecting the 'iPhone' record from becoming an orphan.

Step 4: Reset the Table

To change the rule to allow automatic deletion, we must first remove the current child table.

```
DROP TABLE item;
```

Step 5: Create Child with CASCADE

Now, recreate the table with the **ON DELETE CASCADE** option. This tells Oracle: "If the parent dies, the child dies too."

```
CREATE TABLE item (
    item_id INTEGER PRIMARY KEY,
    item_desc VARCHAR(50) NOT NULL,
    com_id INTEGER,
    -- Foreign Key with Cascade
    FOREIGN KEY (com_id) REFERENCES company_v4(id) ON DELETE CASCADE
);

-- Re-insert the iPhone
INSERT INTO item VALUES (101, 'iPhone', 1);
```

Step 6: Verify the Cascade

Now try the delete operation again. It should succeed this time.

```
-- 1. Delete Apple
DELETE FROM company_v4 WHERE id = 1;

-- 2. Check if the iPhone is still there
SELECT * FROM item;
```

Result: The query should return **no rows** (or only Samsung items). The iPhone was automatically deleted when Apple was deleted.

Key Concept: Foreign Key Actions

When defining foreign key constraints, you can specify what happens when a referenced parent row is deleted. The main options are:

- **RESTRICT/NO ACTION:** Prevent deletion if child rows exist (default behavior)
- **CASCADE:** Automatically delete child rows when parent is deleted
- **SET NULL:** Set foreign key column to NULL when parent is deleted
- **SET DEFAULT:** Set foreign key column to its default value when parent is deleted

For more details, refer to the official documentation:
`https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/create-table-foreign-keys.html`

Part 3: Altering Tables

The `ALTER TABLE` command allows you to modify the structure of an existing table. This section covers BASIC table modification operations.

Key Concept: Prerequisite

In Part 2, we deleted 'Apple' to test the Cascade feature. Before we practice altering tables, let's reset our data so we have a clean slate.

Step 1: Reset Data

Let's bring 'Apple' back and ensure we have both companies ready.

Listing 1: Restoring Data

```
-- Clear existing data to avoid duplicates/conflicts
DELETE FROM item;
DELETE FROM company_v4;

-- Re-insert the companies
INSERT INTO company_v4 VALUES (1, 'Apple');
INSERT INTO company_v4 VALUES (2, 'Samsung');

-- Verify we have 2 rows
SELECT * FROM company_v4;
```

Step 2: Add a Column

Now, let's add a 'country' column. Initially, this column will be empty (NULL) for all rows.

```
ALTER TABLE company_v4 ADD (country VARCHAR2(50));

-- Verify the structure (you will see the new column)
DESCRIBE company_v4;
```

Step 3: Update Data (Crucial Step)

We want to make 'country' a mandatory field (NOT NULL). However, Oracle will not allow this if the column currently contains NULL values. We must fill the data first.

```
-- Fill in the country for Apple
UPDATE company_v4 SET country = 'USA' WHERE id = 1;

-- Fill in the country for Samsung
UPDATE company_v4 SET country = 'Korea' WHERE id = 2;

-- Verify the data looks correct
SELECT * FROM company_v4;
```

Step 4: Add the Constraint

Now that all rows have a country, we can safely enforce the `NOT NULL` constraint.

```
ALTER TABLE company_v4 MODIFY (country NOT NULL);

-- Verify the constraint (Type should act as NOT NULL)
DESCRIBE company_v4;
```

Part 4: Self Exercises (not graded)

Task 1: Create the Tables (DDL)

Write the SQL `CREATE TABLE` statements for the following three tables. You must implement all the specific constraints listed below.

1. **Table: USERS**
   - `user_id`: Integer, Primary Key.
   - `username`: Text, must be **Unique**.
2. **Table: PROJECTS**
   - `project_id`: Integer, Primary Key.
   - `title`: Text.
   - `deadline`: Text (or Date).
   - **Constraint:** Add a `CHECK` constraint to ensure the `title` is at least 3 characters long.
3. **Table: TASKS (The Complex One)**
   - `task_id`: Integer, Primary Key.
   - `description`: Text.
   - `status`: Text.
   - `project_ref`: Integer.
   - `assignee_ref`: Integer.

   Required Logic for TASKS Table:

- **Project Connection:** Link `project_ref` to `PROJECTS(project_id)`.
  - *Requirement:* If a Project is deleted, the Task must be automatically deleted. (**Hint:** Use `ON DELETE CASCADE`).
- **User Connection:** Link `assignee_ref` to `USERS(user_id)`.
  - *Requirement:* If a User is deleted, the Task remains but becomes "Unassigned". (**Hint:** Use `ON DELETE SET NULL`).
- **Status Validation:** The `status` column must ONLY allow three specific values: `'TODO'`, `'DOING'`, or `'DONE'`.
  (**Hint:** Use `CHECK ... IN (...)`).

Task 2: Verification (DML)

Write the SQL statements to prove your logic works. Perform the following steps in order:

1. **Setup Data:** Insert 1 User, 1 Project, and 1 Task (assigned to that User and Project).
2. **Test Status Constraint:** Try to insert a new Task with `status = 'ARCHIVED'`.
   - *Check:* The database should return an error.
3. **Test User Logic:** Delete the User you just created.
   - *Check:* Write a `SELECT` query on the `TASKS` table. The `assignee_ref` should be `NULL`.
4. **Test Project Logic:** Delete the Project you just created.
   - *Check:* Write a `SELECT` query on the `TASKS` table. The task should be gone (empty result).

Task 3: Clean up (DML)

After completing all exercises and verifications, it's important to clean up your database. At the end of your exercise, include the relevant DROP TABLE commands to remove all the tables created during Task 1 and Task 2:

Important Cleanup Note

When dropping tables with foreign key relationships, you must drop the child tables FIRST before dropping parent tables. This avoids foreign key constraint violations.

FYI

Important Reference: Foreign Key Actions

- **RESTRICT** or **NO ACTION**: Prevent deletion of parent row if child rows exist (default)
- **CASCADE**: Automatically delete child rows when parent is deleted
- **SET NULL**: Set foreign key columns to NULL when parent is deleted
- **SET DEFAULT**: Set foreign key columns to their default value when parent is deleted

For complete details, see: `https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/create-table-foreign-keys.html`

End of Side Activity 4.